SOFTWARE REQUIREMENTS SPECIFICATION

Image Caption Translator

Prepared by:

20230359 — Ammr Amr Adawy

20230379 — Omar Moharam Fouad

20230616 — Mina Safwat Fouad

20230477 — Mohamed Saber Ahmed

20230370 — Omar Gaber Sayed Abdel Aal

20230563 — Mustafa Mahmoud Mohamed

Table of Contents

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete, precise, and verifiable requirements for the Image Caption Translator — a cloud-native, microservices-based platform designed to detect textual regions within images, extract the contained text through Optical Character Recognition (OCR), translate that text into a user-specified target language, and render the translated text back onto the original image before storing and returning the processed result. The document is prepared in accordance with IEEE Std 830-1998 and is intended for university evaluation.

1.2 Document Conventions

- SHALL denotes a mandatory requirement.
- SHOULD denotes a strongly recommended, but non-mandatory requirement.
- MAY denotes an optional requirement.
- All API endpoint paths are case-sensitive and must be implemented exactly as specified.
- Database field types are expressed in Java conventions (Long, String, LocalDateTime, etc.).
- Component names in *italics* refer to source files listed in Section 3.

1.3 Intended Audience

This document is primarily addressed to: (a) the development team for implementation guidance; (b) academic evaluators assessing the software engineering methodology; (c) future maintainers requiring an authoritative reference; and (d) test engineers deriving acceptance criteria.

1.4 Project Scope

The Image Caption Translator accepts images containing text in any language supported by the integrated OCR engine and returns a processed image in which the original text has been replaced with its translation. The platform provides a multi-tenant REST API, role-based access control distinguishing standard Users from Administrators, a persistent translation history, a user-facing reporting mechanism, and real-time Server-Sent Events (SSE) notifications. The platform is fully containerised and designed for horizontal scalability.

Out of scope: Native mobile applications, browser extensions, human-in-the-loop translation review, and any language model fine-tuning are explicitly excluded from the current version.

2. Overall Description

2.1 Product Perspective

The Image Caption Translator is a new, standalone system composed of two cooperating microservices: a Spring Boot Backend that exposes a RESTful API and manages persistence, authentication, and business orchestration; and a Python Processing Service (TranslationService) that performs the computationally intensive image analysis pipeline. The two services communicate exclusively through Redis, which acts both as a Pub/Sub event bus and as a durable Stream queue. All components are encapsulated in Docker containers and orchestrated via *docker-compose.yml*.

2.2 Product Functions Summary

ID	Function
F-01	User registration and JWT-based authentication
F-02	Image upload with target-language specification
F-03	Asynchronous image processing pipeline (detection → OCR → translation → rendering → CDN upload)
F-04	Real-time SSE streaming of processing status updates
F-05	Translation history retrieval with pagination
F-06	User-facing report submission for failed translations
F-07	Administrator review and resolution of reports
F-08	Administrator access to system-wide images and user management

2.3 User Classes and Characteristics

2.3.1 Standard User

Registered individuals who upload images and consume translation results. Users interact with the system exclusively through the /Api/User/ namespace. They are expected to have no specialised technical knowledge beyond basic web application usage.

2.3.2 Administrator

Privileged operators who access the /Api/Admin/ namespace to monitor system activity, resolve user-submitted reports, and remove accounts. Administrators are registered via a separate endpoint and hold the *Role.ADMIN* enumeration value.

2.4 Design and Implementation Constraints

- All inter-service communication MUST route through Redis; direct HTTP calls betweenservices are prohibited.
- The database schema MUST NOT deviate from the structure defined in Section 6.
- API endpoint paths MUST match exactly those defined in Section 5.1.
- All secrets (API keys, passwords, JWT signing keys) MUST be supplied via the *.env* file and MUST NOT be hard-coded.
- JWT tokens SHALL be validated by *JwtFilter.java* on every protected route.
- Rate limiting SHALL be enforced by *RateLimitFilter.java* at the API gateway level.
- The Strategy Pattern MUST be employed for OCR, CDN, Translation, and Message Queueadapters.

2.5 Assumptions and Dependencies

- A valid Roboflow API key and model endpoint are available and configured in *.env*.
- A valid Cloudinary account (cloud name, API key, API secret) is available.
- The Redis instance is reachable at the hostname and port specified in *application.properties*.
- The relational database is initialised and its JDBC URL is provided via environment variables.
- Docker and Docker Compose are installed on the deployment host.
- Network egress to Roboflow and Cloudinary APIs is permitted from the Docker host.

3. System Architecture

3.1 High-Level Architecture

The system follows a Microservices Architecture pattern with event-driven inter-service communication. The client interacts solely with the Spring Boot service over HTTPS. Upon receiving an image upload request, the Spring Boot service publishes a processing job to a Redis Stream. The Python TranslationService consumes the job, executes the processing pipeline, and publishes the result back to a Redis Pub/Sub channel. The Spring Boot service subscribes to this channel and delivers the result to the awaiting client via Server-Sent Events. Processed images are stored on Cloudinary (or locally) and their URLs are persisted in the relational database.

3.2 Spring Boot Backend Service

Source root: Translation/src/main/java/com/example/demo/. The service is structured into the following packages:

Package	Key Classes	Responsibility
Aop/	GlobalLoggingAspect.java	Cross-cutting Aspect-Oriented logging for all service-layer method invocations.
config/	CdnConfig, CloudinaryConfig, CorsConfig, MessageQueueConfig, RedisPubSubConfig, SecurityConfig, SwaggerConfig	Spring configuration beans for infrastructure, security, and API documentation.
controller/	AdminController, AuthController, UserController	REST controllers mapping HTTP verbs to service invocations.
DTO/	ErrorResponse, WorkerSchema	Data Transfer Objects for API responses and inter-service message payloads.
Entity/	Images, Reports, UserPrinciple, Users	JPA entity classes representing the database schema.
enums/	Role	Enumeration defining USER and ADMIN role constants.
Exceptions/	GlobalExceptionHandler + 4 domain exceptions	Centralised exception handling returning structured ErrorResponse payloads.
filter/	JwtFilter, RateLimitFilter	Servlet filters for JWT validation and request rate limiting.
repositories/	5 repository interfaces	Spring Data JPA repositories for CRUD and custom query operations.
service/	AdminService, AuthService, EmitterService, JwtService, UserDetailService, UsersService	Business logic layer; EmitterService manages SSE emitter lifecycle.
service/strategy/CdnStrategy/	CdnBase, CloudinaryStrategy	Strategy pattern for CDN provider selection.
service/strategy/MessageQueue/	BaseQueue, RedisStreams, strategy	Strategy pattern for message queue provider selection.

3.3 Python Processing Service (TranslationService)

Source root: TranslationService/. Entry point: *main.py* (Redis consumer loop) and *pipeline.py* (processing orchestrator). The service is organised into strategy sub-packages:

Module / Package	Responsibility
app/processor/renderer.py	Renders translated text onto the source image using OpenCV / Pillow.
app/strategies/broker/	base.py, redis.py, strategy.py — Message broker abstraction; Redis implementation.
app/strategies/cdn/	base.py, cloudinary.py, local.py, strategy.py — CDN upload abstraction with Cloudinary and local filesystem implementations.
app/strategies/detection/	base.py, roboflowdetection.py, strategy.py — Text-region detection via Roboflow object-detection API.
app/strategies/ocr/	base.py, paddleocr.py, strategy.py — OCR abstraction; PaddleOCR implementation.
app/strategies/translation/	base.py, naive.py, strategy.py — Translation abstraction with extensible provider support.
app/utils/image_utils.py	Image loading, resizing, and format conversion helpers.
app/utils/cache.py	Result caching utilities to avoid reprocessing identical jobs.
app/utils/config.py	Environment variable loading and validation.

3.4 Communication Layer — Redis

Redis fulfils two distinct roles within the architecture:

- Redis Streams (job queue): The Spring Boot service appends a *WorkerSchema* JSON payload (image URL, target language, image entity ID) to a named Redis Stream. The Python service maintains a consumer group and processes entries in order, acknowledging each entry upon successful completion. Implemented in *RedisStreams.java* (producer) and *app/strategies/broker/redis.py* (consumer).
- Redis Pub/Sub (result notification): Upon completing the pipeline, the Python service publishes the processed image URL to a Redis channel. The Spring Boot service subscribes via *RedisPubSubConfig.java* and forwards the event to the appropriate SSE emitter managed by *EmitterService.java*.

3.5 Deployment Architecture

The *docker-compose.yml* defines four services: spring-backend, python-processor, redis, and the relational database. All services share an internal bridge network. Environment variables are injected from the *.env* file. The Spring Boot service exposes port 8080 externally; the Python service has no external port exposure. Redis is accessible only within the Docker network.

4. System Features

4.1 Authentication and Authorization

4.1.1 Description

The system SHALL provide stateless JWT-based authentication. Every protected endpoint requires a valid Bearer token in the *Authorization* HTTP header. Role-based access control (RBAC) distinguishes USER and ADMIN roles, enforced by *SecurityConfig.java* and validated in *JwtFilter.java*.

4.1.2 Functional Requirements

ID	Requirement
FR-AUTH-01	The system SHALL allow any anonymous client to register a standard user account via POST /Api/Auth/User/register.
FR-AUTH-02	The system SHALL allow any anonymous client to register an administrator account via POST /Api/Auth/Admin/register.
FR-AUTH-03	The system SHALL authenticate registered users via POST /Api/Auth/login and return a signed JWT on success.
FR-AUTH-04	The system SHALL return HTTP 401 for requests with absent, malformed, or expired JWT tokens on protected routes.
FR-AUTH-05	The system SHALL return HTTP 403 when a USER-role token is presented to an ADMIN-only endpoint, and vice versa.
FR-AUTH-06	The system SHALL expose GET /Api/Auth/health as an unauthenticated liveness probe returning HTTP 200.
FR-AUTH-07	Passwords SHALL be stored as bcrypt hashes; plaintext passwords SHALL NOT be persisted or logged.

4.2 Image Upload and Processing

4.2.1 Description

Authenticated users SHALL upload an image file together with a target language code. The Spring Boot service persists an *Images* entity, enqueues a processing job to Redis Streams, and returns an acknowledgement. The Python service asynchronously executes the five-stage pipeline.

4.2.2 Processing Pipeline

Stage	Name	Description
Stage 1	Detection	Roboflow object-detection API identifies bounding boxes of text regions within the image.
Stage 2	OCR	PaddleOCR extracts raw text strings from each detected bounding box.
Stage 3	Translation	The configured translation strategy translates each extracted string to the target language.
Stage 4	Rendering	renderer.py overlays the translated text onto the image at the original bounding box coordinates.

Stage 5	CDN Upload	The rendered image is uploaded to Cloudinary (or local storage) and the resulting URL is returned.

4.2.3 Functional Requirements

ID	Requirement
FR-IMG-01	The system SHALL accept POST /Api/User/images/upload?targetLang= with a multipart image payload from authenticated USER-role clients.
FR-IMG-02	The system SHALL create an Images entity with image_before set to the original image URL/path and createdAt set to the current timestamp.
FR-IMG-03	The system SHALL publish a WorkerSchema message to the Redis Stream upon successful entity creation.
FR-IMG-04	The Python service SHALL consume the Redis Stream message and execute all five pipeline stages in order.
FR-IMG-05	Upon pipeline completion, the system SHALL update the Images entity with the image_after URL.
FR-IMG-06	The system SHALL return HTTP 400 if the uploaded file is not a recognised image format.
FR-IMG-07	The system SHALL return HTTP 400 if the targetLang query parameter is absent or invalid.

4.3 Translation History Management

4.3.1 Description

Users SHALL be able to retrieve a paginated list of their past translation jobs and delete individual records.

4.3.2 Functional Requirements

ID	Requirement
FR-HIST-01	The system SHALL return a paginated list of the authenticated user's Images entities via GET /Api/User/images/history?page=&size;=.
FR-HIST-02	The system SHALL return HTTP 400 (InvalidPageException) if the requested page index exceeds the total page count.
FR-HIST-03	The system SHALL delete the specified Images entity (and its associated Reports entity via cascade) via DELETE /Api/User/images/{imageId}.
FR-HIST-04	The system SHALL return HTTP 404 (NoDataFoundException) if {imageId} does not belong to the authenticated user.

4.4 Real-Time SSE Streaming

4.4.1 Description

The system SHALL allow authenticated clients to subscribe to a persistent Server-Sent Events connection that delivers processing-completion notifications without polling.

4.4.2 Functional Requirements

ID	Requirement
FR-SSE-01	The system SHALL expose GET /Api/User/images/stream as a text/event-stream endpoint.
FR-SSE-02	EmitterService SHALL register and maintain one SseEmitter per authenticated session.
FR-SSE-03	Upon receiving a result notification on the Redis Pub/Sub channel, the system SHALL push the processed image URL to the corresponding SSE emitter.
FR-SSE-04	The system SHALL complete and remove stale emitters upon timeout or client disconnection.

4.5 Reporting System

4.5.1 Description

Users SHALL be able to submit failure reports against specific translation results. Each report is associated with exactly one Images entity via a one-to-one relationship.

4.5.2 Functional Requirements

ID	Requirement
FR-RPT-01	The system SHALL create a Reports entity linked to the specified image via POST /Api/User/images/{imageId}/reports with failureType and description fields.
FR-RPT-02	description SHALL NOT exceed 1,000 characters; the system SHALL return HTTP 400 on violation.
FR-RPT-03	The system SHALL return HTTP 409 if a report already exists for the specified imageId (unique constraint).
FR-RPT-04	The system SHALL return a paginated list of the authenticated user's reports via GET /Api/User/reports.
FR-RPT-05	The system SHALL delete the specified Reports entity via DELETE /Api/User/reports/{reportId}.
FR-RPT-06	The resolved field SHALL default to false on creation.

4.6 Administration Panel

4.6.1 Description

Administrators SHALL have elevated visibility into system data and the ability to manage user accounts and resolve reports.

4.6.2 Functional Requirements

ID	Requirement
FR-ADM-01	The system SHALL return all reports associated with a specified user via GET /Api/Admin/users/{userId}/reports.
FR-ADM-02	The system SHALL return all Images entities system-wide via GET /Api/Admin/images.
FR-ADM-03	The system SHALL return all unresolved reports (resolved=false) via GET /Api/Admin/reports/unresolved.
FR-ADM-04	The system SHALL return all resolved reports (resolved=true) via GET /Api/Admin/reports/resolved.
FR-ADM-05	The system SHALL delete a user account and all associated data (cascade) via DELETE /Api/Admin/users/{userId}.
FR-ADM-06	All /Api/Admin/** endpoints SHALL return HTTP 403 when accessed by a USER-role token.

5. External Interface Requirements

5.1 REST API Interfaces

The following table enumerates every REST endpoint exposed by the Spring Boot service. All paths are prefixed with the server base URL. Authentication column indicates the required JWT role; 'None' means the endpoint is public.

Method	Path	Auth	Description
POST	/Api/Auth/login	None	Authenticate and receive JWT.
POST	/Api/Auth/User/register	None	Register a standard user account.
POST	/Api/Auth/Admin/register	None	Register an administrator account.
GET	/Api/Auth/health	None	Liveness probe.
POST	/Api/User/images/upload	USER	Upload image for translation (targetLang query param).
GET	/Api/User/images/history	USER	Paginated translation history (page, size params).
GET	/Api/User/images/stream	USER	SSE subscription for processing notifications.
DELETE	/Api/User/images/{imageId}	USER	Delete a translation record by ID.
POST	/Api/User/images/{imageId}/reports	USER	Submit a failure report for an image.
GET	/Api/User/reports	USER	Retrieve the user's submitted reports.

DELETE	/Api/User/reports/{reportId}	USER	Delete a specific report.
GET	/Api/Admin/users/{userId}/reports	ADMIN	Retrieve all reports for a given user.
GET	/Api/Admin/images	ADMIN	Retrieve all images system-wide.
GET	/Api/Admin/reports/unresolved	ADMIN	Retrieve all unresolved reports.
GET	/Api/Admin/reports/resolved	ADMIN	Retrieve all resolved reports.
DELETE	/Api/Admin/users/{userId}	ADMIN	Delete a user account and cascade data.

5.2 Redis Messaging Interface

The *WorkerSchema* DTO defines the message contract between the two services:

```
WorkerSchema { String imageId; // Images entity primary key (stringified Long)
String imageUrl; // Pre-signed or CDN URL of the source image String targetLang;
// ISO 639-1 language code (e.g. "ar", "fr", "de") }
```

- Stream name: Configured in *MessageQueueConfig.java* and *app/utils/config.py*; injected via environment variables.
- Consumer group: Python service registers a single consumer group; Spring Boot acts as the sole producer.
- Result channel: Python publishes to a Redis Pub/Sub channel subscribed by *RedisPubSubConfig.java*.

5.3 CDN / Storage Interface

- Cloudinary (primary): Images are uploaded using the Cloudinary Java SDK (*CloudinaryStrategy.java*) and Python SDK (*cloudinary.py*). Upload responses yield a secure URL stored in *image_after*.
- Local filesystem (fallback): *local.py* stores images in a mounted Docker volume when Cloudinary credentials are absent.
- The CDN strategy is selected at runtime via *CdnConfig.java* and *app/strategies/cdn/strategy.py*.

5.4 Third-Party Services

Service	Protocol	Notes
Roboflow	REST/HTTPS	Text-region object detection; credentials in .env.
PaddleOCR	In-process Python library	Optical character recognition; no external network call required.
Cloudinary	REST/HTTPS (SDK)	Image CDN and persistent storage.
Translation Provider	Configurable (Strategy Pattern)	Text translation; naive.py provides a baseline implementation.

6. Database Design

The relational database schema consists of three entities managed by Spring Data JPA. The schema is authoritative; no additional tables, columns, or relationships exist.

6.1 Entity: USERS

Column	Type	Constraints	Description
id	Long	PK, AUTO_INCREMENT	Primary key.
username	String	UNIQUE, NOT NULL	Unique login identifier.
email	String	NOT NULL	Must be a valid email format; used for identification.
password	String	NOT NULL, min 8 chars	Stored as bcrypt hash.
role	ENUM (Role)	NOT NULL	Values: USER | ADMIN.
permission	String	NULLABLE	Comma-separated permission tokens for fine-grained control.

Relationships: OneToMany → IMAGES (user_id FK); cascade delete removes all associated images.

6.2 Entity: IMAGES

Column	Type	Constraints	Description
id	Long	PK, AUTO_INCREMENT	Primary key.
image_before	String	NULLABLE	URL or path of the original uploaded image.
image_after	String	NULLABLE	URL or path of the rendered, translated image.
createdAt	LocalDateTime	NOT NULL	Timestamp of upload; set automatically.
user_id	Long	FK → USERS.id	Owning user; ManyToOne relationship.

Relationships: ManyToOne → USERS; OneToOne → REPORTS (image_id FK, unique).

6.3 Entity: REPORTS

Column	Type	Constraints	Description
id	Long	PK, AUTO_INCREMENT	Primary key.
failureType	String	NOT NULL	Short classification of the failure (e.g. OCR_FAILURE, TRANSLATION_ERROR).
description	String	NOT NULL, max 1000 chars	Detailed user description of the failure.
createdAt	LocalDateTime	NOT NULL	Timestamp of report submission.
resolved	boolean	NOT NULL, DEFAULT false	True when an administrator has resolved the report.
image_id	Long	FK → IMAGES.id, UNIQUE	Associated image; one-to-one constraint.

Relationships: OneToOne → IMAGES (owning side, unique FK).

6.4 Entity Relationships

A single USERS record may own zero or more IMAGES records (1:N). Each IMAGES record may be associated with at most one REPORTS record (1:1, optional). Deletion of a USERS record cascades to all owned IMAGES records, which in turn cascades to the associated REPORTS record.

7. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The API SHALL return HTTP responses for synchronous endpoints within 500 ms under a load of 50 concurrent users.
NFR-02	Performance	The image processing pipeline SHOULD complete within 30 seconds for images up to 5 MB.
NFR-03	Scalability	The architecture SHALL support horizontal scaling of both microservices independently via Docker Compose scale directives.
NFR-04	Availability	The system SHOULD achieve 99.5% monthly uptime in production; the Redis and database tiers are single points of failure and SHOULD be deployed with replication.
NFR-05	Security	All client-server communication SHALL use HTTPS in production deployments.
NFR-06	Security	JWT tokens SHALL expire within a configurable time window (default: 24 hours); refresh tokens are out of scope.
NFR-07	Security	RateLimitFilter SHALL limit each IP to a configurable number of requests per minute to mitigate denial-of-service attacks.
NFR-08	Security	All sensitive configuration values SHALL be sourced from environment variables; the .env file SHALL NOT be committed to version control.
NFR-09	Maintainability	All service-layer method invocations SHALL be logged by GlobalLoggingAspect.java using AOP, without modification to business logic classes.
NFR-10	Maintainability	New OCR, CDN, translation, or broker providers SHALL be integrable by implementing the corresponding base strategy interface without modifying existing code (Open/Closed Principle).
NFR-11	Observability	The system SHALL expose Swagger/OpenAPI documentation at /swagger-ui.html (configured via SwaggerConfig.java).
NFR-12	Portability	Every service SHALL be deliverable as a self-contained Docker image buildable with the provided Dockerfile; no host dependencies beyond Docker Engine are required.
NFR-13	Reliability	Redis Stream consumer groups SHALL enable at-least-once delivery; the Python service SHALL acknowledge messages only after successful pipeline completion.
NFR-14	Usability	The API SHALL return structured ErrorResponse JSON payloads for all error conditions, including field-level validation messages.
NFR-15	Compliance	CORS policy SHALL be explicitly configured in CorsConfig.java; wildcard origins are acceptable only in development profiles.

8. System Diagrams (Textual Description)

Due to the text-based nature of this document, all diagrams are described in structured prose. Implementers are expected to render formal UML / C4 diagrams from these descriptions using appropriate tooling (e.g. PlantUML, draw.io).

8.1 Context Diagram (C4 Level 1)

External actors: Client (web browser or API consumer) and four external systems: Roboflow, Cloudinary, Relational Database, and Redis. The central system boundary is the Image Caption Translator Platform. The Client communicates with the Platform over HTTPS. The Platform communicates outward to Roboflow (detection), Cloudinary (CDN storage), the Database (persistence), and Redis (messaging).

8.2 Container Diagram (C4 Level 2)

Inside the system boundary there are two containers: Spring Boot API and Python Processor. Supporting infrastructure containers are Redis and Database. The Spring Boot API receives requests from the Client, reads/writes the Database, and produces to the Redis Stream. The Python Processor consumes from the Redis Stream, calls Roboflow and Cloudinary, and publishes results to a Redis Pub/Sub channel. The Spring Boot API subscribes to the Pub/Sub channel and pushes results to the Client via SSE.

8.3 Sequence Diagram — Image Upload and Processing

Step	Actor	Action
1	Client → Spring Boot	POST /Api/User/images/upload?targetLang=ar (multipart image, JWT header)
2	Spring Boot	Validates JWT; saves image_before to CDN; creates Images entity (image_after=null)
3	Spring Boot → Redis Stream	XADD WorkerSchema{imageId, imageUrl, targetLang}
4	Python Processor → Redis Stream	XREADGROUP: consumes WorkerSchema message
5	Python Processor → Roboflow	POST detection request; receives bounding boxes
6	Python Processor	PaddleOCR extracts text from each bounding box
7	Python Processor	Translation strategy translates extracted strings
8	Python Processor	renderer.py overlays translated text onto image
9	Python Processor → Cloudinary	Uploads rendered image; receives CDN URL
10	Python Processor → Redis Pub/Sub	PUBLISH result channel {imageId, imageAfterUrl}
11	Spring Boot (subscriber)	Receives Pub/Sub message; updates Images entity (image_after = CDN URL)
12	Spring Boot → Client (SSE)	Sends SSE event with imageAfterUrl on GET /Api/User/images/stream

8.4 Class Diagram Overview — Spring Boot

Central entity classes (*Users*, *Images*, *Reports*) are annotated with JPA annotations. Each entity has a corresponding Spring Data repository. Service classes (*UsersService*, *AdminService*, *AuthService*) depend on repositories and on strategy interfaces (*CdnBase*, *BaseQueue*). Controllers depend on service classes only. *JwtService* is used by both *AuthService* and *JwtFilter*.
EmitterService is used by the Pub/Sub listener registered in *RedisPubSubConfig*.

8.5 Strategy Pattern — Python Service

Each strategy family follows the identical pattern: a *base.py* abstract class defines the interface; one or more concrete classes implement it; a *strategy.py* factory selects the implementation based on environment configuration. The four strategy families are: Broker (Redis), CDN (Cloudinary / local), Detection (Roboflow), and OCR (PaddleOCR). The Translation family follows the same structure with *naive.py* as the baseline implementation.

9. Deployment

9.1 Docker Compose Architecture

The root *docker-compose.yml* declares all four services. Build contexts point to the respective *dockerfile* files in each service directory. An internal bridge network named *translation-net* connects all containers.

Service	Build / Image	Port Mapping	Notes
spring-backend	Translation/dockerfile	8080:8080	Spring Boot API; depends on redis and database.
python-processor	TranslationService/dockerfile	None	Python pipeline; depends on redis.
redis	redis:7-alpine (official)	Internal only	Message broker; no external port exposed.
database	postgres:15 or mysql:8 (configurable)	Internal only	Relational persistence; volume-mounted data directory.

9.2 Environment Variables (.env)

The *.env* file at the repository root supplies runtime configuration to both services. The following variables MUST be defined:

Variable	Consumed By	Description
DB_URL	Spring Boot	JDBC connection string for the relational database.
DB_USERNAME	Spring Boot	Database user name.
DB_PASSWORD	Spring Boot	Database password.
JWT_SECRET	Spring Boot	Base64-encoded HMAC-SHA256 signing key (min 256 bits).
JWT_EXPIRATION_MS	Spring Boot	Token validity duration in milliseconds.
REDIS_HOST	Both	Redis hostname within the Docker network.
REDIS_PORT	Both	Redis port (default: 6379).
REDIS_STREAM_NAME	Both	Name of the Redis Stream used for job publishing.
CLOUDINARY_CLOUD_NAME	Both	Cloudinary account cloud name.
CLOUDINARY_API_KEY	Both	Cloudinary API key.
CLOUDINARY_API_SECRET	Both	Cloudinary API secret.
ROBOFLOW_API_KEY	Python	Roboflow API key for the detection model.
ROBOFLOW_MODEL_ID	Python	Roboflow project / model identifier.
CDN_STRATEGY	Both	Selects CDN provider: 'cloudinary' | 'local'.
TRANSLATION_STRATEGY	Python	Selects translation provider (e.g. 'naive').

9.3 Build and Run Instructions

• Clone the repository and create a *.env* file at the root, populating all variables listed in Section 9.2.

• Execute: docker compose up --build from the repository root.

• The Spring Boot API will be available at http://localhost:8080.

• Swagger UI is accessible at http://localhost:8080/swagger-ui.html.

• To scale the Python processor: docker